It is possible for objects to behave like arrays. One advantage of using an object as an array is that PHP handles objects differently from other data types: objects are not copied on assignment (unlike arrays). As in earlier chapters, we're jumping ahead and doing some object-oriented programming. If the following examples are just completely baffling, you could always have a sneak preview of the first couple of sections in Chapter 6, "Object Fundamentals."

The main benefit of `ArrayObject` over arrays is that it's possible to modify an `ArrayObject` in different parts (scopes) of your application without having to keep track of references yourself (Chapter 3, "Functions," contains a full explanation of lexical scope). For example:

```
$guitarsObj = new ArrayObject([
    ['make' => 'Ibanez', 'model' => 'RG-8570z'],
]);

$guitarsArr = [
    ['make' => 'Ibanez', 'model' => 'RG-8570z'],
];

addGuitar($guitarsObj);
addGuitar($guitarsArr);

print_r($guitarsObj);
print_r($guitarsArr);

// type of $arr must be either array or ArrayObject
function addGuitar (array|ArrayObject $arr) {
    $arr[] = [
        'make' => 'Fender',
        'model' => 'American Ultra Stratocaster'
    ];
}

Output:
ArrayObject Object
(
    [storage:ArrayObject:private] => Array
        (
            [0] => Array
```